

Introducción a la Ciencia de Datos - Tarea 2

Grupo 3

Abu Arab, Juan
5.041.245-7

Quincke, Santiago
5.094.032-3

25 de junio de 2026

1. Código fuente

El código de la tarea se encuentra en el siguiente repositorio de Github: <https://github.com/SQuincke/intro-cd>.

2. Parte 1

2.1. Parte 2

Para visualizar el balance de artículos de cada medio en los conjuntos de train y test, se utilizó un gráfico de barras como se puede ver en la Figura 1.

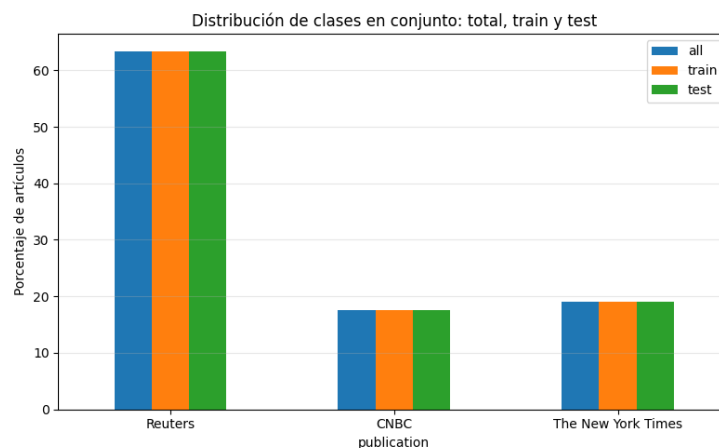


Figura 1: Balance de clases para conjuntos: train, test y total

Se observa que se mantiene el balance de clases en los conjuntos de train y test, siendo las proporciones de aproximadamente 63 % (Reuters), 18 % (CNBC), 19 % (The New York Times) del total de cada conjunto (que es la misma proporción previa a la separación en conjuntos de train y test).

2.2. Parte 3

Bag of Words (BoW) es una de las técnicas más sencillas de procesamiento de lenguaje natural, la cual implica convertir un texto en un vector de enteros que lo represente, donde cada posición representa una palabra (o n-grama) y el valor representa la cantidad de ocurrencias en el texto de esa palabra.

Como ejemplo simplificado, supongamos que tenemos un corpus de dos documentos:

- Documento 1: «el gato está sentado»
- Documento 2: «el perro está sentado con el gato»

Primero, extraemos cada palabra presente y le asignamos una posición en el vector de features: [el, gato, está, sentado, perro, con]

Luego, contamos la cantidad de ocurrencias de cada palabra en el texto y lo representamos en el vector de features:

- Documento 1: [1, 1, 1, 1, 0, 0]
- Documento 2: [2, 1, 1, 1, 1, 1]

De esta forma, obtenemos una representación vectorial del texto.

En ejemplos más prácticos donde se tiene un corpus de texto de varios documentos, si para cada documento se ubica su vector de features en una fila de una matriz, se obtiene una matriz de dimensiones $n \times k$, donde n es la cantidad de documentos en el corpus y k es la cantidad de palabras distintas en el corpus. Como generalmente cada documento contiene una proporción chica de las palabras del corpus de texto, se obtiene una matriz donde la mayoría de entradas son 0 (sparse matrix).

En el ejercicio se obtuvo una matriz de dimensiones 10252×85435 .

2.3. Parte 4

Un n -grama representa una “subsecuencia de n palabras contiguas en una secuencia de texto”. Dada una secuencia de texto de k palabras, la cantidad posible de n -gramas (siendo $n \leq k$) es:

$$k - n + 1$$

TF-IDF es un algoritmo de extracción de features de texto que premia a los elementos por aparecer muchas veces en un documento, pero castiga a las palabras que aparecen frecuentemente a través del conjunto de documentos. Como resultado, se obtiene un valor para cada elemento en cada documento (con la misma dimensión que la representación Bag of Words obtenida anteriormente).

La forma en que se calcula es mediante el producto $TF \times IDF$. TF representa la frecuencia de aparición del elemento i en el documento j , y se calcula como:

$$TF(i, j) = \frac{\#apariciones_del_elemento_i}{\#elementos_del_documento_j}$$

Por otro lado, IDF se calcula como:

$$IDF(i) = \log \left(\frac{\#cantidad_de_documentos}{\#cantidad_de_documentos_en_los_que_aparece_el_elemento_i} \right)$$

De esta forma, el valor final de TF-IDF se obtiene multiplicando ambas medidas. Como resultado, obtenemos una matriz de la mismas dimensiones que la obtenida en BoW.

2.4. Parte 5

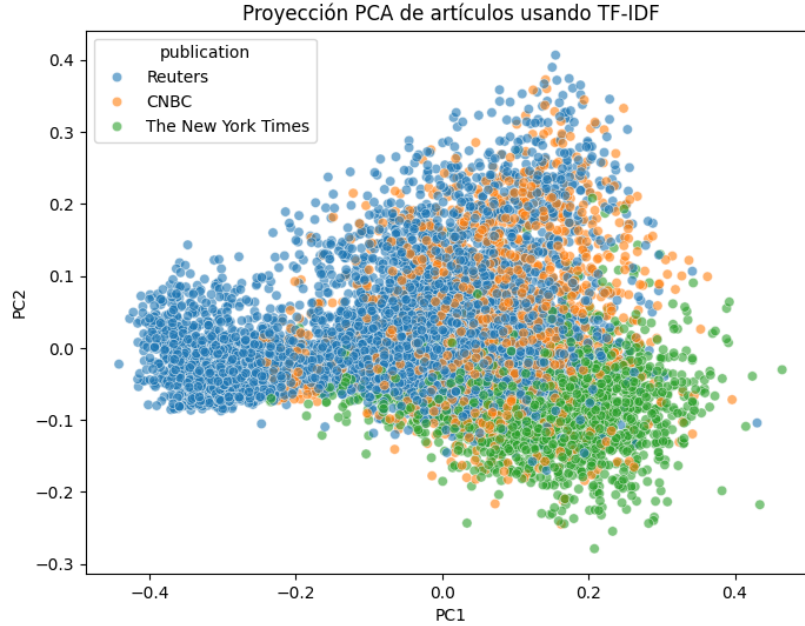


Figura 2: Proyección dataset train usando primeros dos componentes PCA

Para entender de una forma más matemática qué efectos tiene usar stopwords (más allá de eliminar palabras), se buscó una matriz de transformación A tal que:

$$X_{PCA_2} \approx X_{PCA_1} A$$

donde X_{PCA_1} corresponde a la proyección obtenida utilizando la representación TF-IDF completa y X_{PCA_2} a la proyección con stopwords.

La matriz A se estimó mediante mínimos cuadrados, obteniéndose:

$$A = \begin{bmatrix} -0,3751 & 0,0145 \\ 0,0341 & 0,5944 \end{bmatrix}$$

Los coeficientes fuera de la diagonal son cercanos a cero, lo que indica que ambas componentes principales mantienen aproximadamente la misma orientación y presentan poca mezcla entre sí.

Por otro lado, el coeficiente $-0,3751$ asociado a la primera componente principal indica un cambio de signo como se ve en la Figura 3 acompañada de una compresión. De forma similar, el coeficiente $0,5944$ asociado a la segunda componente principal indica una compresión de dicha componente, aunque conservando su orientación original.

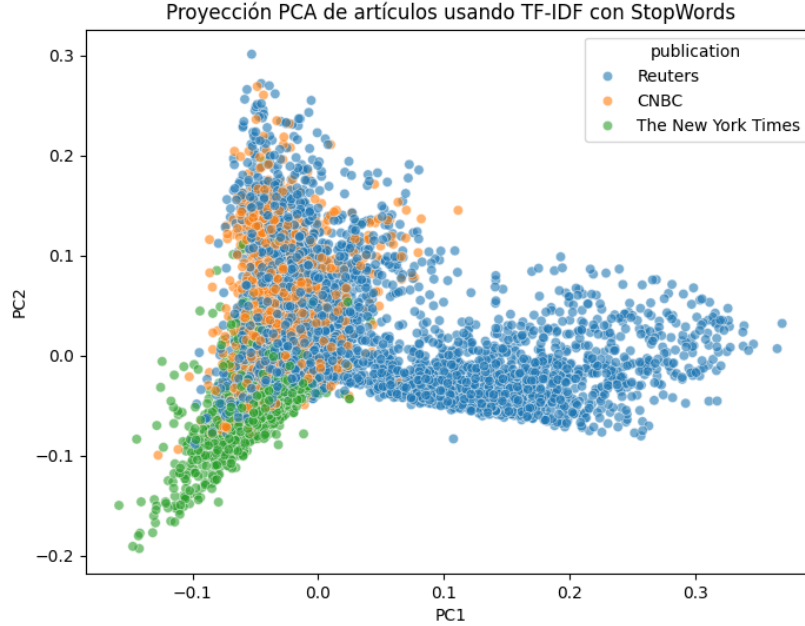


Figura 3: PCA con stop words

En consecuencia, el filtrado de características no produjo una separación de clases significativa del espacio proyectado, sino principalmente una reducción de escala en las componentes principales y una inversión de la primera componente.

Se evaluaron distintas configuraciones del vectorizador TF-IDF. Entre ellas, las que produjeron mayores cambios fueron aquellas que modificaban el rango de los *n-gramas*. Debido a limitaciones de hardware, se utilizó **TruncatedSVD** en lugar de PCA. Al utilizar bigramas o trigramas se observó una distribución diferente de los documentos en el espacio proyectado; sin embargo, la separación visual entre editoriales en $\mathbb{R}^2$ siguió siendo limitada.

También se analizaron las palabras y frases más representativas de cada configuración. Para ello se emplearon *n-gramas* de mayor longitud junto con un filtrado por frecuencia mínima de aparición (**min_df**), con el objetivo de reducir el vocabulario a un conjunto manejable y eliminar términos poco informativos. Esto permitió identificar expresiones recurrentes y características de los documentos, aunque dichas diferencias no resultaron suficientes para generar grupos claramente separados en la proyección bidimensional.

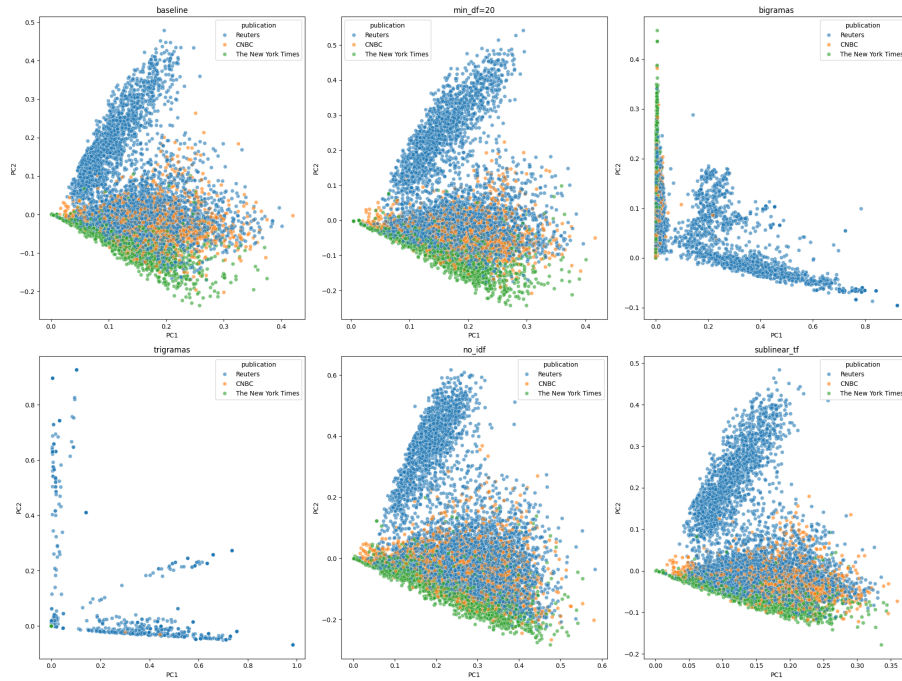


Figura 4: Comparación de TF-IDFs

Se intentó identificar qué palabras tenían mayor “peso” en cada editorial. Idealmente, cada medio se distribuye en una dirección distinta en el espacio reducido, por ejemplo:

$$\text{NYT} \rightarrow (-1, -1), \quad \text{Reuters} \rightarrow (1, 0), \quad \text{CNBC} \rightarrow (0, 1)$$

A partir de esta representación, se seleccionaron los documentos más alejados del origen $(0, 0)$, ya que estos corresponden a textos más “representativos” de cada clase. Luego, dentro de esos documentos extremos, se analizaron las palabras con mayor frecuencia (o mayor contribución en TF-IDF) para caracterizar cada editorial.

Los resultados muestran que, mientras *The New York Times* tiende a enfatizar temas políticos y figuras presidenciales, Reuters presenta una mayor concentración de términos asociados a economía. Sabiendo que CNBC reponea artículos de Reuters, es lógico ver que las nubes de puntos correspondientes se superpongan.

Palabras con mayor peso por editorial

Se listan las palabras con mayor peso para cada editorial, obtenidas a partir de los documentos más alejados del origen en el espacio PCA.

Reuters

Palabra	Peso
million	0.1495
yuan	0.1394
million yuan	0.1348
euros	0.1128
million euros	0.1039
net profit	0.0945
net	0.0912
profit	0.0801
fy	0.0708
versus	0.0676
year ago	0.0674
profit million	0.0628
euros year	0.0619
yuan million	0.0572
source text	0.0563
company coverage	0.0559
text chinese	0.0547
text	0.0543
chinese	0.0540
coverage	0.0532

CNBC

Palabra	Peso
percent	0.1019
fed	0.0479
dollar	0.0475
oil	0.0365
index	0.0353
earnings	0.0321
markets	0.0316
market	0.0306
higher	0.0302
crude	0.0296
futures	0.0290
data	0.0283
rates	0.0282
bank	0.0275
year	0.0271
shares	0.0264
trade	0.0254
rate	0.0252
growth	0.0250
stocks	0.0249

The New York Times

Palabra	Peso
mr	0.2644
mr trump	0.1702
trump	0.1504
president	0.0501
said	0.0390
house	0.0303
campaign	0.0282
republican	0.0267
ms	0.0252
impeachment	0.0243
clinton	0.0242
mueller	0.0210
democrats	0.0208
white house	0.0207
white	0.0198
mr biden	0.0180
mr mueller	0.0173
biden	0.0168
kushner	0.0157
political	0.0156

Vemos que, debido a la gran cantidad de características generadas por la representación TF-IDF, la varianza explicada por cada componente principal individual es relativamente baja. Esto apunta a que la información se encuentra distribuida en muchas dimensiones y que no existe una única dirección dominante que capture una gran proporción de la variabilidad de los datos.

Además, se observa que a partir de la componente principal 15 el aporte individual de varianza pasa a ser inferior al 1 %. Sin embargo, aunque el aporte individual sea reducido, las componentes posteriores pueden seguir contribuyendo de manera acumulativa a la varianza total explicada. Por este motivo, la selección del número de componentes no debería basarse únicamente en el aporte individual, sino también en la varianza acumulada y en los objetivos del análisis.

La ausencia de un “codo” pronunciado en la curva de varianza acumulada sugiere que la estructura de los datos no puede resumirse eficazmente en unas pocas dimensiones. Este comportamiento es habitual cuando se trabaja con textos, donde la información suele estar dispersa entre numerosos términos y combinaciones de términos.

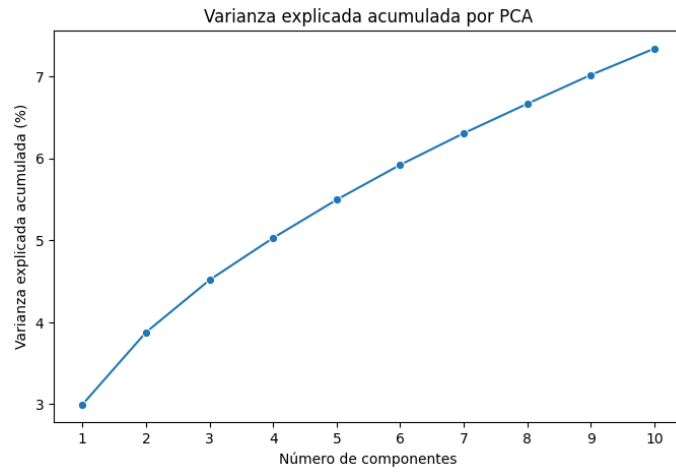


Figura 5: Varianza explicada acumulada por PCA

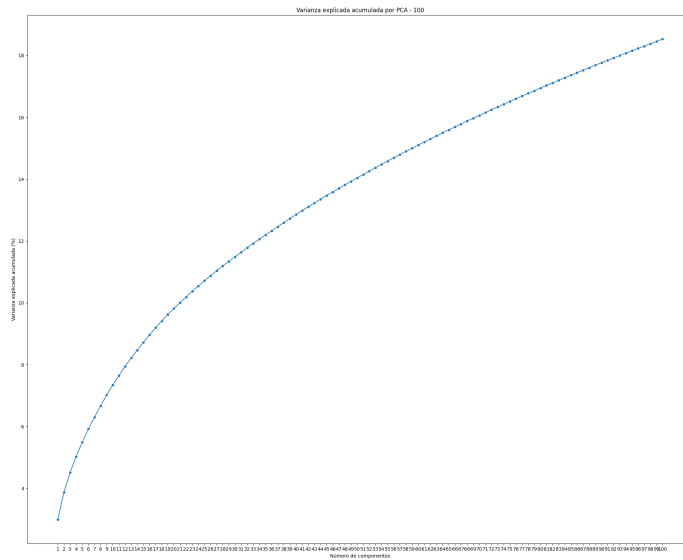


Figura 6: Varianza explicada acumulada por PCA (100 componentes)

3. Parte 2

3.1. Parte 1

Se entrenó un modelo *Multinomial Naive Bayes* con el split de las top 3 editoriales más representadas. Los resultados obtenidos sobre el conjunto de

prueba se muestran en la Figura 7 y en el Cuadro 1. Se obtuvo una accuracy de 0,70. Teniendo en cuenta la distribución observada previamente en los datos, era esperable que el modelo obtuviera un mejor desempeño al clasificar los artículos de Reuters y un peor desempeño al clasificar los artículos de CNBC. Los resultados obtenidos confirman este comportamiento esperado.

Si se considera únicamente la *accuracy*, se está observando solamente la proporción total de predicciones correctas realizadas por el modelo. Sin embargo, esta métrica no brinda información sobre qué tipos de errores se están cometiendo ni sobre el desempeño particular en cada clase. Por este motivo, suele complementarse con métricas como *precision*, *recall* y *F1-score*, que permiten analizar con mayor detalle la calidad de las predicciones.

Además, en este conjunto de datos las clases no se encuentran balanceadas, por lo que la *accuracy* puede resultar engañosa. En situaciones de este tipo, un modelo puede alcanzar una accuracy relativamente alta simplemente favoreciendo a la clase más frecuente, sin necesariamente aprender a distinguir correctamente entre todas las categorías. Por ejemplo, si una editorial concentra la mayoría de los ejemplos, un clasificador que predijera siempre esa clase obtendría una accuracy considerable, aunque su utilidad práctica sería limitada.

Por esta razón, para evaluar adecuadamente el desempeño del modelo resulta necesario considerar conjuntamente la matriz de confusión y las métricas por clase, ya que estas ofrecen una visión más completa de los aciertos y errores del clasificador.

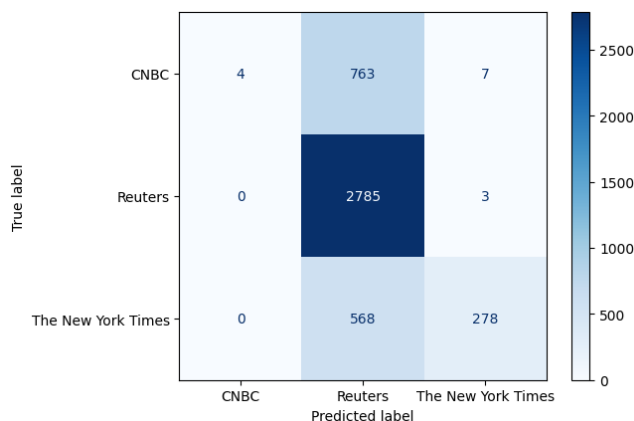


Figura 7: Matriz de confusión

Clase	Precision	Recall	F1-score	Support
CNBC	1.00	0.01	0.01	774
Reuters	0.68	1.00	0.81	2788
The New York Times	0.97	0.33	0.49	846
Macro avg	0.88	0.44	0.44	4408
Weighted avg	0.79	0.70	0.61	4408

Cuadro 1: Resultados de clasificación utilizando Multinomial Naive Bayes.

La *recall* y la *precision* se calculan utilizando los valores de verdaderos positivos (TP), falsos positivos (FP) y falsos negativos (FN). Sus fórmulas son:

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

La *recall* mide qué proporción de los ejemplos positivos reales fue correctamente identificada por el modelo, mientras que la *precision* mide qué proporción de las predicciones positivas realizadas por el modelo fue correcta. Viendo la matriz de confusión, los verdaderos positivos (TP) de la clase i se encuentran en la posición M_{ii} , ya que corresponden a los ejemplos de la clase i que fueron correctamente clasificados como pertenecientes a dicha clase.

Los falsos positivos (FP) corresponden al resto de la columna i , es decir, ejemplos que fueron predichos como pertenecientes a la clase i , pero que en realidad pertenecen a otra clase.

Los falsos negativos (FN) corresponden al resto de la fila i , es decir, ejemplos que realmente pertenecen a la clase i , pero que fueron clasificados como pertenecientes a otra clase.

Por último, los verdaderos negativos (TN) corresponden a todas las entradas que no pertenecen ni a la fila i ni a la columna i , representando ejemplos de otras clases que tampoco fueron clasificados como la clase i .

3.2. Parte 2

La validación cruzada (cross-validation) es una técnica utilizada para estimar de forma más robusta el desempeño de un modelo sobre datos no vistos. Consiste en realizar varias divisiones del conjunto de entrenamiento en subconjuntos de entrenamiento y validación, entrenando y evaluando el modelo repetidamente sobre distintas particiones. En el caso particular de *k-fold cross-validation*, el conjunto se divide en k particiones o *folds*, y en cada iteración una de ellas se utiliza para validación mientras las restantes se usan para entrenamiento.

Esta técnica puede utilizarse tanto para comparar modelos como para seleccionar hiperparámetros. El conjunto de test, en cambio, no debe emplearse durante la selección del modelo ni el ajuste de hiperparámetros, ya que eso introduciría sesgo en la evaluación. Su uso debe reservarse para la evaluación final del modelo seleccionado.

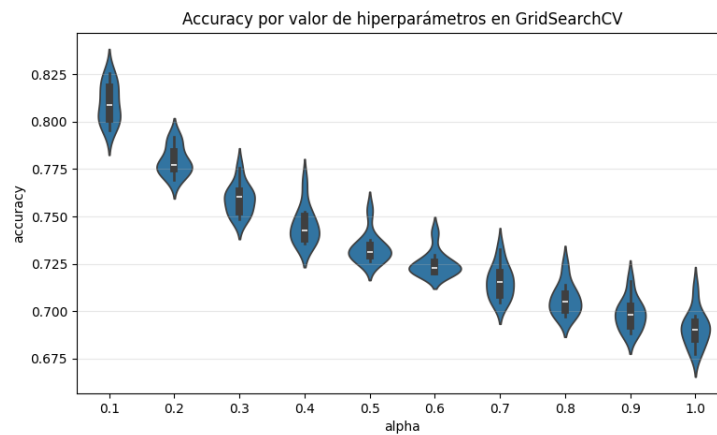


Figura 8: Accuracy por alpha

3.3. Parte 3

En el problema concreto se realizó una búsqueda de hiperparámetros sobre el “alpha” en Naive Bayes, teniendo como función objetivo la “accuracy”. Vemos en la Figura 8 que con un alpha menor los resultados son mejores. Luego se entrenó un NB con este alpha, dando un modelo mejor que el entrenado originalmente, este es capaz de reconocer mas artículos de CNBC, que antes no podía.

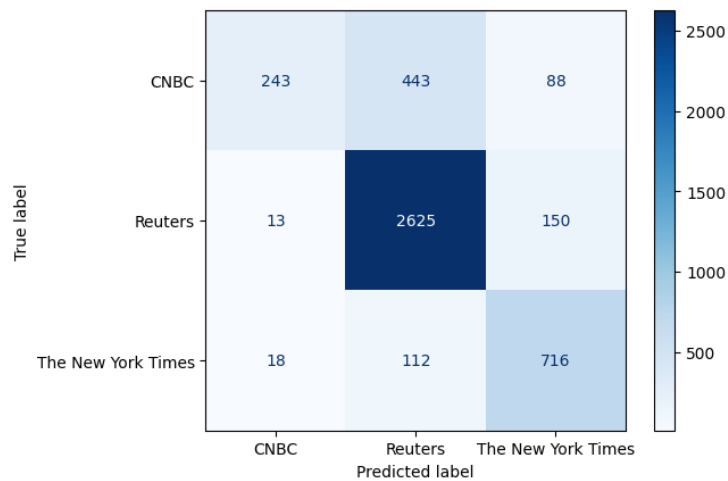


Figura 9: Matriz de Confusion del Mejor Modelo

Clase	Precision	Recall	F1-score	Support
CNBC	0.89		0.31	0.46
774				
Reuters	0.83	0.94	0.88	2788
The New York Times	0.75	0.85	0.80	846
Macro avg	0.82	0.70	0.71	4408
Weighted avg	0.82	0.81	0.79	4408

Cuadro 2: Resultados de clasificación del Mejor Modelo.

Accuracy obtenida: 0.81

Vemos que el modelo mejora sustancialmente respecto al original, pudiendo clasificar mejor artículos de CNBC.

3.4. Parte 4

De decidió usar Regresión Logística como modelo alternativo. Este modelo estima, para cada clase, la probabilidad de pertenencia a partir de una combinación lineal de las características de entrada. A diferencia de Multinomial Naive Bayes, no asume independencia entre las palabras. Los hiperparámetros del modelo fueron ajustados utilizando cross validation. La regresión logística multiclase funciona de manera similar a la regresión lineal en una primera etapa. Dado un vector de características x , se calcula un puntaje lineal

$$z = w^T x + b,$$

Posteriormente, estos puntajes se transforman en probabilidades mediante la función *softmax*:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}},$$

donde K es la cantidad total de clases. El resultado es un vector de probabilidades de longitud K , donde cada componente representa la probabilidad estimada de pertenecer a una clase.

Una propiedad importante de la función *softmax* es que los componentes del vector siempre suman 1:

$$\sum_{k=1}^K p_k = \sum_{k=1}^K \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = \frac{\sum_{k=1}^K e^{z_k}}{\sum_{j=1}^K e^{z_j}} = 1.$$

Por eso, la salida puede interpretarse como una distribución de probabilidad de las distintas clases.

Los resultados obtenidos se pueden ver la Figura 10 y el Cuadro 3.

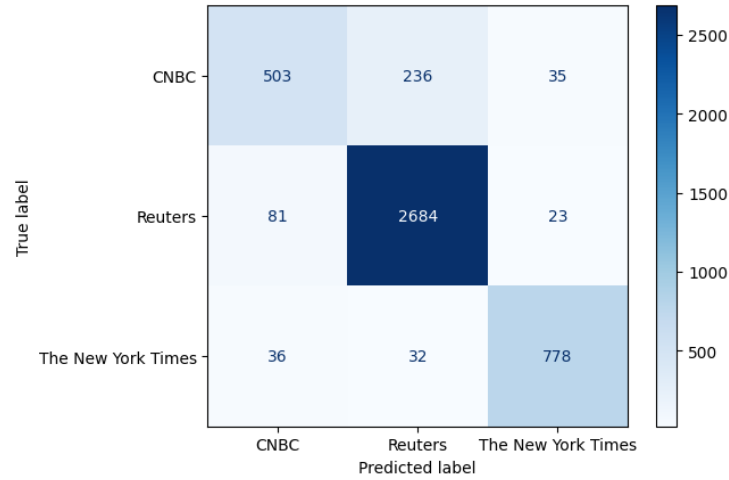


Figura 10: Matriz de Confusion RL

Clase	Precision	Recall	F1-score	Support
CNBC	0.81	0.65	0.72	774
Reuters	0.91	0.96	0.94	2788
The New York Times	0.93	0.92	0.93	846
Macro avg	0.88	0.84	0.86	4408
Weighted avg	0.90	0.90	0.90	4408

Cuadro 3: Resultados de clasificación usando Regresion Logistica.

Accuracy obtenida: 0.90

Se puede observar una mejora en casi todas las métricas respecto a los modelos entrenados anteriormente.

3.5. Parte 5

Se descartó la editorial CNBC y se agregó la editorial Vox. Esto hizo que la distribución quedara aún más desbalanceada que la original (ver Figura 11). Reuters representa más del 70 % de los datos, por lo que un modelo que prediga que todos los artículos pertenecen a Reuters obtendría una accuracy cercana a 0.7.

Existen varias formas de mitigar el problema de la distribución desigual entre clases, entre ellas el submuestreo (*undersampling*) y el (*oversampling*). El sobre-muestreo consiste en aumentar la cantidad de ejemplos de las clases minoritarias, ya sea duplicando datos, creando nuevos sintéticos, o aplicando augmentations a las clases con menor cantidad de datos. Esto hace que el modelo dé mayor atención a las clases minoritarias durante el entrenamiento.

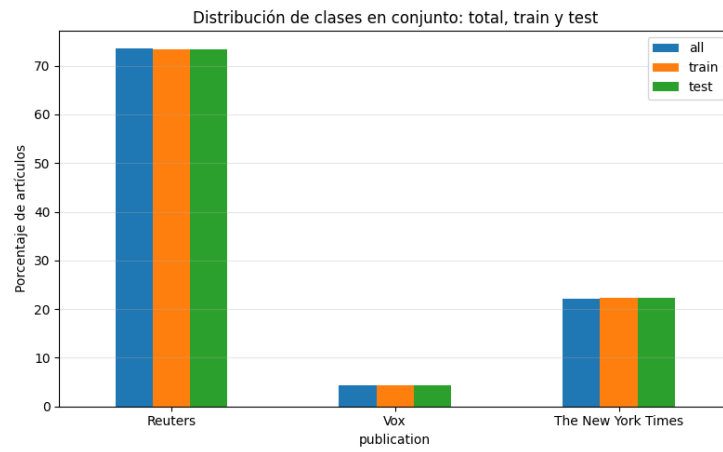


Figura 11: Balance de clases para conjunto Top2 & Vox

El submuestreo consiste en eliminar ejemplos de las clases mayoritarias, reduciendo su tamaño. Esto permite obtener una distribución más equilibrada entre las clases, aunque a costa de perder información.

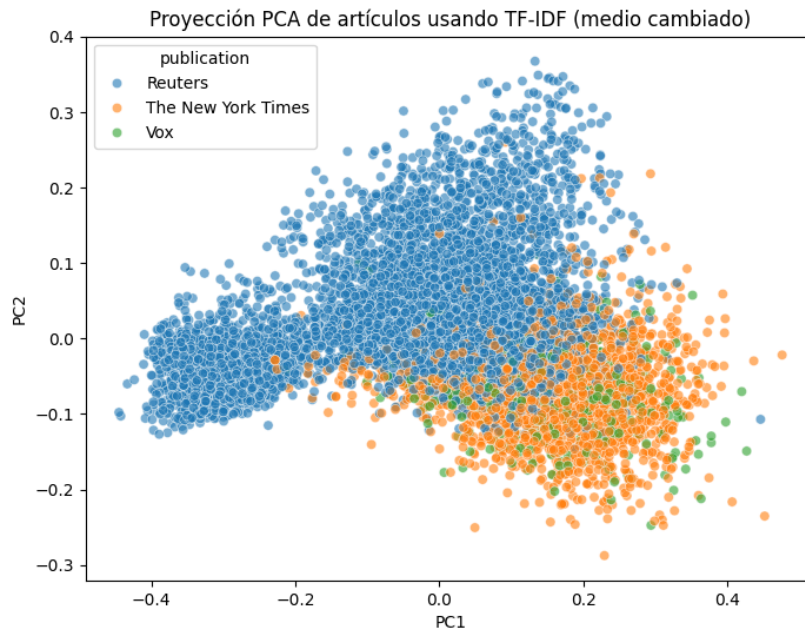


Figura 12: Proyección PCA Top2 & Vox

Al entrenar un modelo *NB* sobre este nuevo *split*, se observa un desempeño muy deficiente para *Vox*. El modelo parece overfitear a *Reuters* y, a *The New York Times*), ignorando por completo los ejemplos de *Vox*.

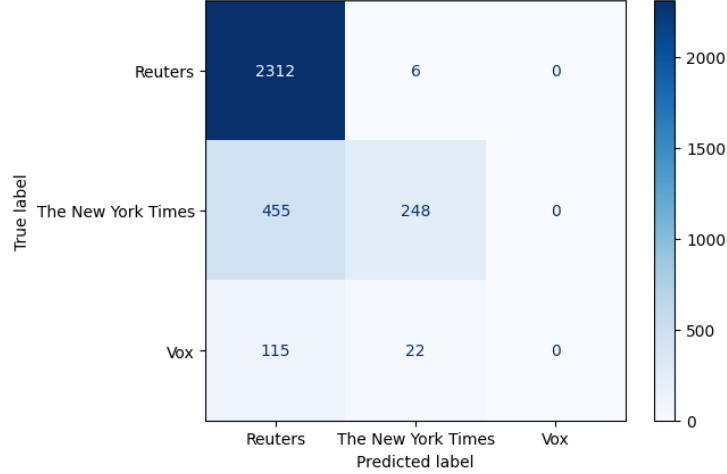


Figura 13: Matriz de Confusion Top2 & Vox

Clase	Precision	Recall	F1-score	Support
Vox	0.00	0.00	0.00	137
Reuters	0.80	1.00	0.89	2318
The New York Times	0.90	0.35	0.51	703
Macro avg	0.57	0.45	0.47	3158
Weighted avg	0.79	0.81	0.77	3158

Cuadro 4: Resultados de clasificación para Top2 & Vox

Accuracy: 0.90

3.6. Parte 6

Además de TF-IDF existen otros métodos para representar texto. Un ejemplo es el uso de *word embeddings*. En lugar de representar cada documento mediante un vector de features utilizando n-gramas del dataset, a cada palabra se le asigna un vector de dimensión fija llamado *embedding*.

De esta forma, un documento pasa a ser representado como una matriz de dimensiones $N \times d$, donde N es la cantidad de palabras del documento y d la dimensión del *embedding*, este ultimo es estandar para todas las palabras,

Esto genera el problema de que distintos documentos tienen diferentes tamaños de entrada. Para utilizar algoritmos clásicos de aprendizaje automático,

una alternativa es obtener una única representación por documento promediando los *embeddings* de sus palabras. Otra posibilidad es aplicar *padding*, agregando valores de relleno hasta alcanzar una longitud fija.

Algunos métodos populares para obtener *embeddings* son Word2Vec, GloVe y FastText. Estos permiten capturar relaciones semánticas entre palabras, algo que representaciones como Bag of Words o TF-IDF no modelan de forma explícita.

3.7. Parte 7

Se buscó entrenar un modelo Naive Bayes utilizando únicamente el título de los artículos en lugar del cuerpo del texto. El modelo tiende a predecir la clase Reuters la mayor parte de las veces (14), lo que sugiere que la información contenida en los títulos no es suficiente para discriminar adecuadamente entre los distintos medios. Se concluye que, al contener una mayor cantidad de información, el cuerpo del artículo resulta más identificatorio que su título.

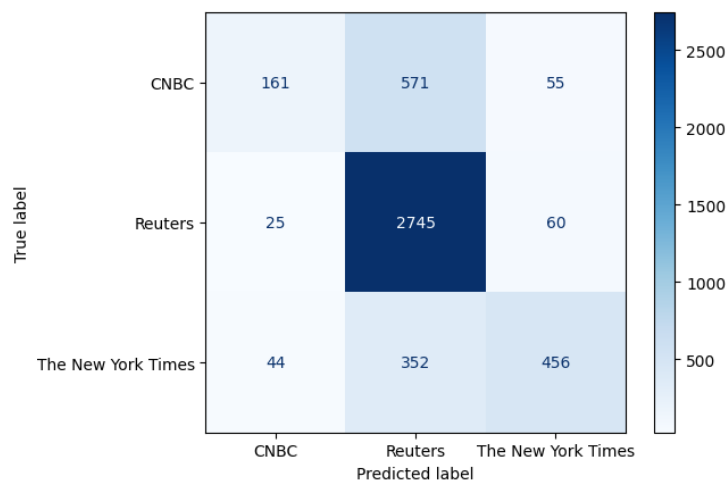


Figura 14: Matriz de Confusion para titulos

Clase	Precision	Recall	F1-score	Support
CNBC	0.70	0.20	0.32	787
Reuters	0.75	0.97	0.84	2830
The New York Times	0.80	0.54	0.64	852
Macro avg	0.75	0.57	0.60	4469
Weighted avg	0.75	0.75	0.71	4469

Cuadro 5: Resultados de clasificación para Titulos.

Accuracy obtenida: 0.75

A. Uso de IA

Se usaron herramientas de inteligencia artificial (ChatGPT) como apoyo para la revisión del informe. Se limitó a la corrección de errores gramaticales y ortográficos, y a sugerencias menores en el código implementado, principalmente relacionadas con la mismatch de shapes y errores de memoria.

Las herramientas que se usaron no participaron en la generación del contenido, el análisis de resultados ni la elaboración de las conclusiones. Todas las contribuciones intelectuales y decisiones presentadas en este trabajo corresponden al grupo.